# NANOGRAMA 5X5

LÓGICA PARA CIENCIAS DE LA COMPUTACIÓN

**JUAN SEBASTIÁN PEDRAZA**
**TOMAS FELIPE PATIÑO**
**LUCETH ARGOTE RADILLO**

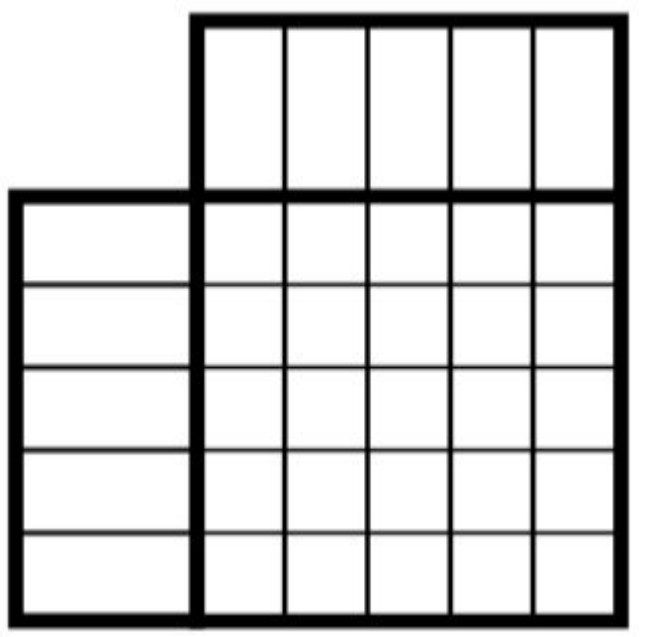
# CONTENIDO

- PROBLEMA
- POSIBLE CASO
- CLAVES DE REPRESENTACIÓN
- REGLAS DEL JUEGO

# PROBLEMA

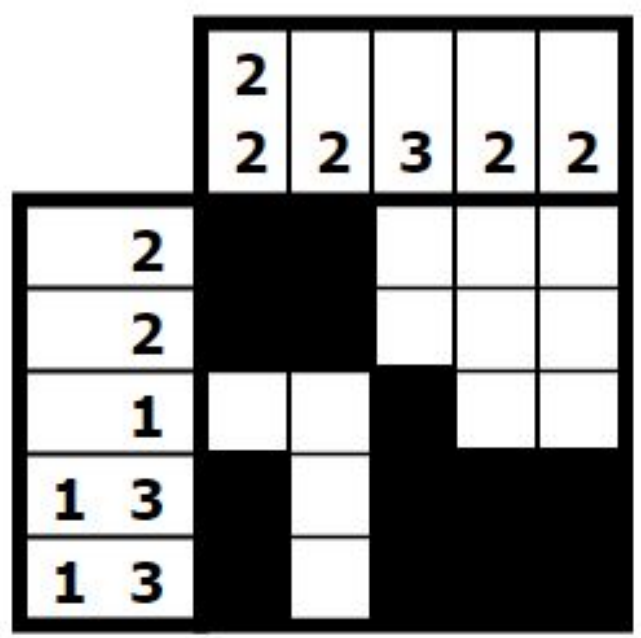
- Explicación:

Considere un nanograma 5x5. El problema consiste en rellenar los cuadros según los números indicados sobre cada columna y fila cumpliendo así las reglas del juego.



# POSIBLE CASO

- Observe el siguiente nomograma que está completado y cumple las reglas del juego



# CLAVES DE LA REPRESENTACIÓN

- Enumeramos cada casilla del nanograma 5x5 de la siguiente manera:

| | | | | | |
|---|---|---|---|---|---|
| | 0,0 | 0,1 | 0,2 | 0,3 | 0,4 |
| | 1,0 | 1,1 | 1,2 | 1,3 | 1,4 |
| | 2,0 | 2,1 | 2,2 | 2,3 | 2,4 |
| | 3,0 | 3,1 | 3,2 | 3,3 | 3,4 |
| | 4,0 | 4,1 | 4,2 | 4,3 | 4,4 |

# CLAVES DE LA REPRESENTACIÓN

- Para representar las 25 letras proposicionales se emplearon descriptores proposicionales de la siguiente manera :

Ren(n,m) = La casilla (n,m) está rellena

| | | | | | |
|---|---|---|---|---|---|
| | 0,0 | 0,1 | 0,2 | 0,3 | 0,4 |
| | 1,0 | 1,1 | 1,2 | 1,3 | 1,4 |
| | 2,0 | 2,1 | 2,2 | 2,3 | 2,4 |
| | 3,0 | 3,1 | 3,2 | 3,3 | 3,4 |
| | 4,0 | 4,1 | 4,2 | 4,3 | 4,4 |

# MODELO DE REPRESENTACIÓN

Ren(0,1): Casilla 0,1 rellena

¬Ren(0,2): Casilla 0,2 rellena

¬Ren(0,3): Casilla 0,3 rellena

¬Ren(0,4): Casilla 0,4 rellena

¬Ren(2,0): Casilla 2,0 rellena

¬Ren(3,0): Casilla 3,0 rellena

Ren(2,0): Casilla 2,0 rellena

# REGLAS DEL JUEGO

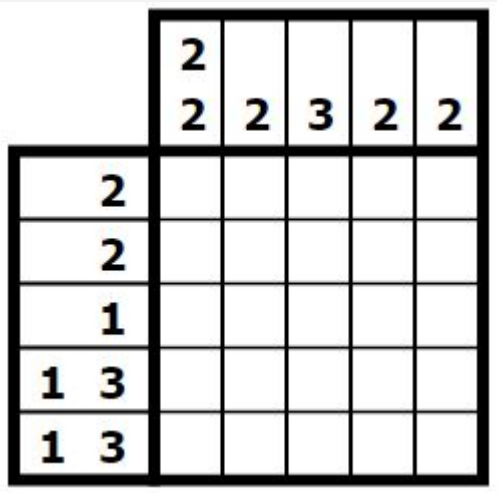
- Una de las claves para generalizar el problema a cualquier nomograma es ser consciente de la variabilidad de las reglas dado que según el número indicado en la parte superior de cada fila/columna estas varían. Lo que no varía es la cantidad de reglas, de este modo:

- Regla 1 :  regla para fila 0
- Regla 2:  regla para fila 1
- …..
- Regla 5: regla para fila 4

- Regla 6: regla para columna 0
- Regla 7: regla para columna 1
- …
- Regla 10:  regla para columna 4

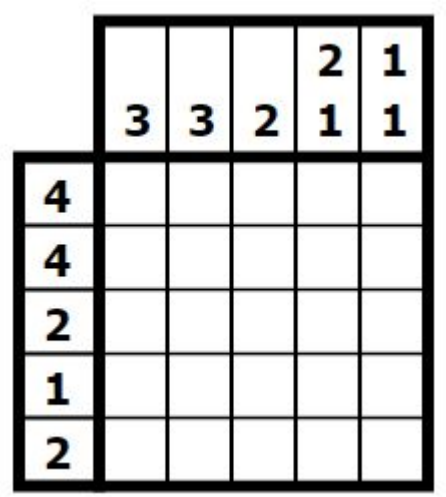
# REGLAS DEL JUEGO

- Por ejemplo suponga los dos siguientes casos de Nanogramas




- Para este caso:

Regla 1: dos casillas rellenas consecutivas en la fila 0

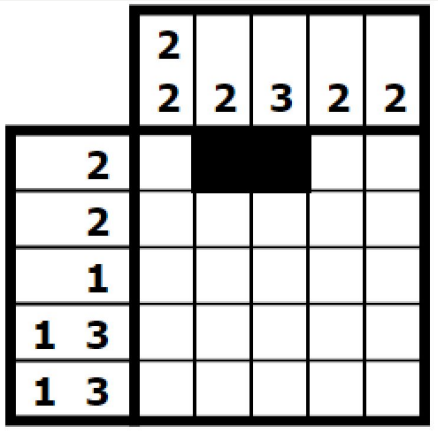



- Para este caso:

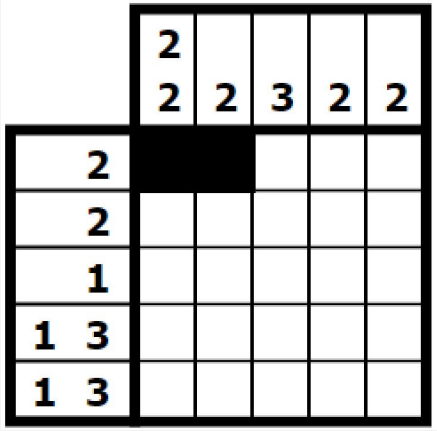
Regla 1: cuatro casillas rellenas consecutivas en la fila 1

# EJEMPLO REGLA 1

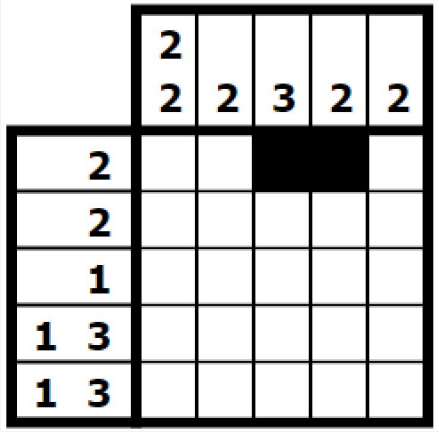
- Si hay 2 casillas rellenas consecutivas en la fila 0 tenemos varias opciones:



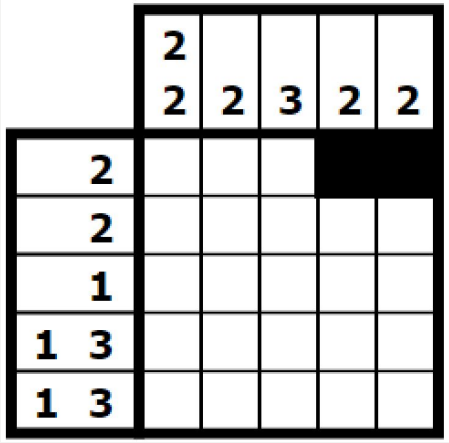

Opción 4




Opción 1




Opción 2




Opción 3

# GENERALIZACION REGLAS

- De este modo en lenguaje proposicional obtenemos la siguiente formula para la regla 1 del caso anterior

$$\bigvee_{i=0}^{3}\left[Ren_{(0,i)} \wedge Ren_{(0,i+1)} \bigwedge_{j=o,j\neq i,i+1}^{4}\left(\neg Ren_{(0,j)}\right)\right]$$

- Para cualquier fila n tenemos

$$\bigvee_{i=0}^{3}\left[Ren_{(n,i)} \wedge Ren_{(n,i+1)} \bigwedge_{j=o,j\neq i,i+1}^{4}\left(\neg Ren_{(n,j)}\right)\right]$$

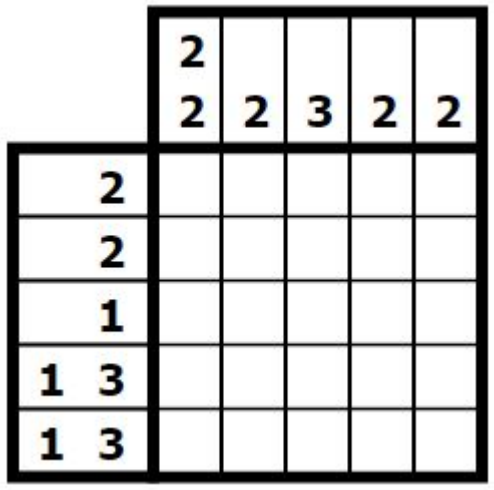
- Para cualquier columna n tenemos

$$\bigvee_{i=0}^{3}\left[Ren_{(i,n)} \wedge Ren_{(i+1,n)} \bigwedge_{j=o,j\neq i,i+1}^{4}\left(\neg Ren_{(j,n)}\right)\right]$$

NOTA: Observar que al momento de generalizar para cualquier fila n es el número de la fila y para cualquier columna solo cambia de órdenes el índice del descriptor

# REGLAS DEL JUEGO

- De este modo definimos veintiséis reglas que son todas las posibles condiciones iniciales de cada fila y columna. De este modo el nanograma solo tiene 10 reglas que es una por cada fila y columna




```python
self.condiciones_iniciales =  {x : None for x in range(10)}

self.condiciones_iniciales[0] = 2
self.condiciones_iniciales[1] = 2
self.condiciones_iniciales[2] = 1
self.condiciones_iniciales[3] = [1,3]
self.condiciones_iniciales[4] = [1,3]


self.condiciones_iniciales[5] = [2,2]
self.condiciones_iniciales[6] = 2
self.condiciones_iniciales[7] = 3
self.condiciones_iniciales[8] = 2
self.condiciones_iniciales[9] = 2
```

Condiciones iniciales en Python